Get unlimited access to the best of Medium for less than $1/week. Become a member

Containerization Machine Learning Applications



Salman Anwaar Follow 4 min read · Sep 24, 2024

Open in app

Medium Search Write





Containerizing a Machine Learning (ML) project involves packaging the entire ML workflow, including code, libraries, and dependencies, into containers to ensure consistency across different environments (development, testing, and production). Different types of containers can be used for specific tasks in the ML lifecycle, such as training, inference, batch prediction, and hybrid containers. Let's dive into each of these:

Training-only Containers:

Purpose: Containers solely responsible for training the machine learning model.

- **Input**: Training data and hyperparameters.

- **Output:** Trained model artifacts (e.g., `.h5`, `.pkl`, `.pt` files).

Steps:

1. **Package the training script:** This script contains all the necessary steps to train the model (data loading, preprocessing, training logic, hyperparameters tuning).
2. **Define dependencies:** List all required libraries (TensorFlow, PyTorch, Scikit-learn, etc.) in a `requirements.txt` or `environment.yml` file.
3. **Create Dockerfile:** Define the environment for the training process, including base image, libraries, and commands to run the training script.
4. **Build and run the container:**

- Use Docker to build the container image.
- Run the container by passing the training data and other configurations as environment variables or mounting external volumes for datasets

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . /app
CMD ["python", "train.py"]
```

Scaling: You can scale these containers horizontally using orchestration platforms like Kubernetes to train the model with distributed datasets or hyperparameter tuning.

Inference-only Containers:

Purpose: Containers that serve the trained model for making predictions (inference).

- **Input:** Trained model artifact and new data for prediction.
- **Output:** Predictions.

Steps:

1. **Deploy trained model:** Use the model artifact from the training container.
2. **Expose a web service:** Use a framework like **Flask** or **FastAPI** to serve the model as an API endpoint.
3. **Create Dockerfile:** Define the necessary environment and the web service that loads the trained model and exposes an API for inference.
4. **Build and deploy the container:** Build the container and expose the prediction service.

Dockerfile Example:

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . /app
CMD ["python", "app.py"]
```

API service: The `app.py` could be a Flask service that loads the model and exposes `/predict` for requests.

```
from flask import Flask, request, jsonify
import pickle

app = Flask(__name__)
model = pickle.load(open('model.pkl', 'rb'))

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json(force=True)
    prediction = model.predict([data['input']])
    return jsonify({'prediction': prediction[0]})
```

Scaling: Use load balancing for high-volume inference services by deploying multiple inference containers with tools like Kubernetes or AWS Fargate.

Batch Prediction Containers:

Purpose: Containers that perform inference on a large dataset in batches.

- **Input:** Large dataset (e.g., CSV file, database).
- **Output:** Predictions for all data points.

Steps:

1. **Model loading:** Similar to inference containers, but designed to handle large datasets.
2. **Batch processing:** Modify the script to read large batches of data from a file or database, process the data, and make predictions in bulk.

3. **Data storage:** Store the results in an output file (e.g., CSV or database).
4. **Dockerfile setup:** Similar to inference containers but tailored for batch processing.

Dockerfile Example:

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . /app
CMD ["python", "batch_predict.py"]
```

Batch prediction script (`batch_predict.py`):

```
import pandas as pd
import pickle# Load model
model = pickle.load(open('model.pkl', 'rb'))
# Load input data in batches
data = pd.read_csv('input_data.csv', chunksize=1000)
# Process and predict
for chunk in data:
    predictions = model.predict(chunk)
    chunk['predictions'] = predictions
    chunk.to_csv('output_predictions.csv', mode='a')
```

Scaling: Container orchestration tools can distribute the batch prediction task across several containers, reducing processing time.

Hybrid Containers:

Purpose: Containers that handle training, batch prediction, and inference within a single container.

- **Input**: Depends on the operation (training: dataset, inference: new data).
- **Output**: Trained model or predictions.

Steps:

1. **Combine functionalities**: The container will have logic for:

- Training the model.
- Saving/loading the trained model.
- Serving an API for real-time inference.
- Performing batch predictions if needed.

2. Dockerfile setup: The same Dockerfile will include instructions to install dependencies for training and inference.

3. Command-based execution: The container can execute different functionalities based on passed commands or environment variables.

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . /app

ENTRYPOINT ["python"]
CMD ["main.py"]
```

```python
import sys

def train():
    print("Training model...")
    # Model training logic here

def batch_predict():
    print("Running batch predictions...")
    # Batch prediction logic here

def serve_inference():
    print("Serving inference API...")
    # Serve inference logic here

if __name__ == "__main__":
    if sys.argv[1] == 'train':
        train()
    elif sys.argv[1] == 'batch_predict':
        batch_predict()
    elif sys.argv[1] == 'serve_inference':
        serve_inference()
```

Use case: For smaller projects or when simplicity is required, hybrid containers provide flexibility by combining training, batch prediction, and real-time inference in one container.

Pipeline Containers: this is what we're going to implement

Purpose: Each phase of the ML pipeline (data ingestion, preprocessing, feature extraction, training, and evaluation) is containerized separately.

- **Steps:**

1. **Containerize each pipeline stage:** Each stage (e.g., data preprocessing, feature engineering) has its own container.

2. **Orchestration:** Use tools like **Kubernetes** or **Apache Airflow** to orchestrate the pipeline by triggering different stages sequentially or in parallel.
3. **Scaling:** Each stage can scale independently, which is useful for heavy tasks like data preprocessing or model training.

Benefits:

- Flexibility in scaling different stages independently.
- Modular design allows easy updates to individual pipeline components.

Conclusion:

The containerization of ML projects introduces flexibility, scalability, and consistency. Depending on the specific needs of your workflow (training, real-time inference, batch prediction, etc.), you can choose from various container types. Hybrid containers offer an all-in-one solution, while pipeline containers are more suitable for complex, multi-stage workflows.

Kubernetes | Machine Learning | Mlops | Docker | Python



Written by Salman Anwaar

77 followers · 15 following

Data And AI Enthusiast

Follow

No responses yet

Baolongnguyen Mac

What are your thoughts?

More from Salman Anwaar



Salman Anwaar

AWS SageMaker Studio vs AWS Canvas: Choosing the Right Tool f...

Introduction: AWS's Machine Learning Suite —Empowering Every User

Oct 31, 2024 3



Salman Anwaar

Which tool is better? SageMaker Studio Vs SageMaker Notebook...

When you are signed in to your AWS account and you are in the AWS services page, you...

Aug 22, 2021 30 1



Salman Anwaar

Explore Databricks Data Storage: A Quick 1-Minute Read to Elevate...

In Databricks, various data storage components exist, each serving specific...

Dec 29, 2023 14



Salman Anwaar

Explore Databricks Scope of Library Installation: A Quick Read...

In Databricks, you have various options to install libraries, and they can be scoped to...

Jan 1, 2024

See all from Salman Anwaar

Recommended from Medium





Fahad Ahammed

4 Important Development Practice to Make Your Python Application...

These are not only for Python, will fit for other stacks as well.

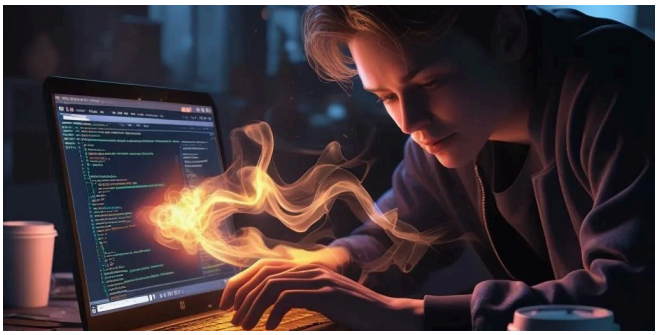
Jun 1 31 1

Gary Bao

Smarter Sampling: Data Sampling Strategies That Make or Break Yo...

In machine learning, how to sample your training data?

Jun 29



Suresh Kandru

From Chaos to Clarity: How I Built My First Real MLOps Project (And...

A beginner-friendly guide to organizing your machine learning experiments without losin...

Aug 4



Pablo Pazos

Why Everyone Is Migrating from MLflow to Weights & Biases (W&B...

“The tooling you use defines the speed at which you innovate.”

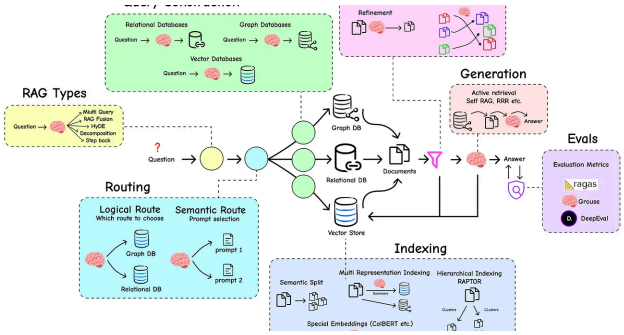
May 15 13



In Artificial Intelligence in Plain En... by Mehmet Ö...

Vector Databases Explained: Why AI Needs Memory for the Real...

Learn what vector databases are, why they’re essential for AI, and how they power...



In Level Up Coding by Fareed Khan

Building the Entire RAG Ecosystem and Optimizing Every Component

Routing, Indexing, Retrieval, Transformation and more.